

The Same Zero: Why Identical ASR Can Imply Different Guarantees in LLM-Agent Security

Anonymous ACL submission

Abstract

LLM-agent security has produced a dense landscape of defenses—prompt hardening, content filters, permission gates, sandboxes, provenance firewalls—each claiming to make agents “safe.” We argue that the field lacks a pre-purchase question: *what does a given defense actually guarantee, and where does that guarantee come from?* We apply Verification Autonomy Levels (VAL, arXiv:2608.19009), a taxonomy that grades verification/authorization schemes by the source of their spec (L0: LLM self-declaration, no deterministic anchor; L1: deterministic rules; L2: objective ground truth, correctness only; L3/L4: decidable systems with single-property or domain-level completeness; L5: impossible), to 22 agent-security defenses. The classification is a falsifiable predictor: a defense fails the way its anchor fails. We validate this with frozen prediction cards on a small published sample (10/10 hits, flagged as such) and then run the first controlled deployment-value comparison: same budget, two stacks—a VAL-guided stack (intent-anchored confirmation gate + schema sandbox) versus a mainstream intuition stack (prompt hardening + keyword filter)—across 50 scenarios, 12 attack families (including third-party AgentDojo payloads), real tool effects, adaptive, white-box and PAIR-style attacks, and three seeds (~7,000 LLM calls). The VAL stack holds 0.000 attack success with 1.000 benign success in every condition; the intuition stack also reaches 0.000 ASR but kills all benign actions and its security is model-behavior luck (compliance 0.235), not structure (the VAL stack tolerates 2.4x more model compromise—0.567 compliance—and still blocks everything). The same zero, two different guarantees—zero is an outcome, not a guarantee. Under escalating attacks, zero stability becomes a measurable axis: some zeros collapse abruptly, others erode gradually, and indistinguishable stability trajectories

can rest on opposite guarantees (model behavior vs. platform record). VAL is a guarantee-provenance framework for LLM-agent security: it identifies what an observed security outcome is grounded in, and hence where it will fail. We further identify where L3 anchors exist in agent security: confinement and information-flow properties (sandboxing, taint tracking, data-control separation), not unrestricted semantic safety, which caps at L2 (restricted semantic properties that can be formally encoded remain eligible for higher levels).

1 Introduction

Large language model (LLM) agents are moving from single-turn assistants to persistent systems that call tools, browse the web, and maintain long-term memory—and the matching expansion of security defenses (prompt hardening, filters, permission gates, sandboxes, provenance firewalls) is presented with implicit, rarely commensurable guarantees: one paper’s attack success rate (ASR) is not another’s, and no framework tells a deployer *which defense to trust for which threat, before running experiments*.

This paper asks the question the field has not asked explicitly: **what does a given agent-security defense guarantee, and where does that guarantee come from?**

We answer with Verification Autonomy Levels (VAL), a taxonomy proposed in (Yin 2026) for verification schemes generally. VAL classifies any verification/authorization scheme along a single axis—the source of its verification spec—into six levels:

- **L0:** the spec is LLM self-declaration (“I checked it,” “this is safe”). No deterministic anchor. Failure mode: the declaration can be rewritten by the very model that issues it.

- **L1:** deterministic rules (regex, keyword lists, hand-written policies). Failure mode: surface forms are bypassable by rewriting.
- **L2:** objective ground truth (gold labels, platform-recorded events, execution results). Guarantees *correctness only*: everything checked passes, but nothing is proven about what was not checked (the completeness blind spot).
- **L3/L4:** decidable systems (solveset equivalence, type systems, formal kernels, confinement mechanisms). Single-property or domain-level completeness within an operational design domain (ODD).
- **L5:** universal completeness—impossible in the unrestricted case.

VAL’s core claim is that a scheme’s level is determined by its *anchor*, not by its accuracy or sophistication, and that the anchor determines the scheme’s failure modes. Applied to agent security, this yields a falsifiable prediction: **a defense is defeated the way its anchor fails**. An L0 defense fails when the model complies with the injection; an L1 defense fails when the attacker rewrites around the surface rule; an L2 defense fails outside its ODD (forged metadata, poisoned data); an L3 defense holds within its ODD by construction.

We make three contributions:

1. **A level map of agent security.** We classify 22 defenses using a frozen, blind-rater-validated protocol (inter-rater $\kappa \approx 0.8$ (Yin 2026)), organizing the landscape by what each defense can actually guarantee.
2. **Prediction validation.** Frozen level-to-behavior cards, validated on a small published sample (10/10 hits, flagged; two mechanism-driven corrections)—the point is falsifiability, not the hit count.
3. **Deployment-value comparison and zero stability.** Same budget, two stacks (VAL-guided vs. mainstream intuition), one testbed—50 scenarios, 12 attack families, real effects, adaptive/white-box/PAIR escalation, three seeds, a second victim, and the official AgentDojo benchmark. The VAL stack dominates on security and utility; escalating-attack trajectories ($Z(\alpha)$) further separate defenses by failure morphology (abrupt collapse, progressive erosion, flat behavioral vs. structural plateau), and iden-

tical trajectories still require provenance to interpret—zero stability joins ASR and compliance as a reportable property.

What this paper claims—the spine in one paragraph.

The argument runs on three levels. *Phenomenon:* two agent-security defenses can reach the same observed zero ASR with opposite guarantees—visible in the compliance gap and across two victim models (Section 6). *Mechanism:* the difference lies in the verification anchor, i.e., where the security verdict is grounded (Section 5). *Method:* VAL provides a compact language for identifying the strongest guarantee a defense actually grounds, and predicts the failure boundary of each level. A meta-claim completes the picture: the classifier used here is itself an L1 tool, and we measured rather than assumed its reproducibility (Section 8). If a reader remembers only one sentence, it should be: *the same zero is not the same guarantee—zero is an outcome, not a guarantee*.

What we do not claim. VAL is not a general theory of AI verification, a complete security evaluation, or a ranking of defenses by quality. It is a mid-level methodological claim about LLM-agent security: when two defenses produce the same observed security outcome, the provenance of that outcome—which verification anchor grounds it—determines its guarantee and predicts its failure boundary. Claims about robotics, formal methods, human-in-the-loop systems, or non-agentic LLMs are outside this paper’s scope; so is a claim that VAL covers every possible defense. We study the question the field has not asked explicitly, within the setting where we can answer it with controlled experiments and third-party benchmarks.

2. Background and Related Work

2.1 Verification Autonomy Levels (VAL)

VAL (Yin 2026) grades verification schemes by three questions: Q1 *where does the verification spec come from* (LLM declaration / deterministic rule / objective truth / decidable system); Q2 *what does the verdict guarantee* (nothing / correctness / completeness); Q3 *what scope does a completeness claim cover* (single property / domain / universal). The level is the first rule that fires: L5 (universal completeness, impossible), L4 (domain completeness), L3 (single-property completeness), L2 (correctness with objective anchor), L1 (correctness with deterministic rule), L0 (otherwise).

The procedure is deterministic and has been measured for inter-rater reproducibility: three independent blind raters reach $\kappa \approx 0.8$ on 48–70 schemes under the refined protocol (Yin 2026).

Two refinements matter for security. First, **anchor semantics** (Zhang et al. 2026): objective anchors split by *what they certify*—intent (a recorded decision event: “the user confirmed this target”), truth (a fact: “the answer matches ground truth”), effect (an execution outcome: “the action actually occurred”). A confirmation record is an intent anchor; it authorizes, but says nothing about whether the outcome was correct. Conflating intent with effect is the category error behind most authorization failures. Second, **the completeness blind spot** (Yin 2026): substitution- and sampling-based checks verify proposed candidates but cannot prove that no candidate was missed. In security, the analog is the *un-enumerated attack*: a gate that blocks all evaluated attacks is a correctness probe over the evaluated distribution, not a guarantee that no attack path was missed.

2.2 Agent-security defenses and their taxonomies

The defense literature is large. Text-level defenses (prompt hardening (Zhou et al. 2023), keyword/content filters, Llama Guard (Inan et al. 2023)) operate on the current context; tool-level defenses (allowlists (Chen et al. 2023), permission systems (Shi et al. 2025), information-flow control (Costa et al. 2025), sandboxes) gate execution; memory-level defenses (A-MemGuard (A-MemGuard 2025), PPMF (Xu et al. 2026)) protect persistent state; benchmarks (AgentDojo (Debenedetti et al. 2024)) measure attack success and utility trade-offs. Recent work has begun to move beyond single-defense evaluation: Injection-Execution Dissociation (*Injection-Execution Dissociation* 2026) shows that injection success and tool-execution success are separable safety properties (memory storage rates >97.5% while downstream execution ranges 0–95%, uncorrelated); the Source-of-Authority SLR (*LLM-Based Test Oracles* 2026) classifies LLM test oracles by where their authority comes from and finds over half reach verdicts with no specification at all; When Does Verification Pay Off (Lu 2025) shows that same-family verification yields near-zero gain while cross-family verification remains valuable. Our contribution is different in kind: we do not

add another defense, we provide a *pre-purchase axis*—the anchor—that organizes the entire landscape and predicts each defense’s failure modes, and we measure whether choosing by this axis beats choosing by intuition.

3 Method

3.1 Classification protocol

We classify defenses with the frozen VAL protocol (v2.3, (Yin 2026)), extended for security: R1 benchmarks are N/A (not runtime verifiers); R2 mixed systems are split per component with the weakest anchor on the verdict path reported; R3 evidence anchors do not equal verdict anchors (retrieved documents are L2 evidence; an LLM alignment verdict over them is L0); R4 silver anchors (deterministic recomputation without gold) are L1; R10 designer-authored unvalidated rules are L1; R12b theorem-backed exhaustive procedures (confinement, taint tracking) are decidable (L3); R15 LLM-generated verification expectations place the weakest anchor at L0. Each defense is rated on Q1/Q2/Q3 from a mechanism-only evidence pack by blind raters; classifications reported here are the committed (weakest-anchor) levels. For the security subset (22 agent-security items), two independent blind raters agree exactly on 20/22 (90.9%); the two disagreements are known boundary cases (an LLM-alignment verdict classified under rule R3, and a single-property vs domain completeness scope).

On the validity of the taxonomy. A taxonomy cannot be validated by its own consistency: agreement among raters shows the categories are *usable*, not that they are *right*. We offer three independent lines of evidence instead. First, *out-of-sample prediction*: the level-to-failure-mode cards (Section 3.2) were frozen before outcome data and hit 10/10 on published results, with two mechanism-driven corrections—retrospective storytelling cannot produce that pattern. Second, *cross-benchmark convergence*: the same anchor logic predicts our own testbed results (Section 6), AgentDojo’s official numbers (Section 6.5), JADE’s MCP instances (Section 6.5), and LITMUS’s OS-level behavior (Section 7)—four independent sources with no shared implementation. Third, *falsifiability*: each card names the specific attack predicted to break its defense, so a single well-chosen counterexample refutes the mapping. This is an operational defense: we claim VAL is

a productive organizing principle, and we make it easy to disprove—not that it is the unique correct ontology of agent security.

3.2 Prediction protocol

Following the reverse-validation discipline of (Yin 2026), we freeze prediction cards before consulting outcome data: for each defense, level $\rightarrow$ predicted behavior pattern (ASR regime, bypass mechanism, benign-cost mode) + the specific attack predicted to break it. Cards are signed and dated, then validated against published data (PPMF’s own numbers; abstract-level evidence for other systems, flagged as such). The protocol also generalizes out of sample: before running the third-party JADE MCP instances (Section 6.5), we froze five predictions (e.g., N’s keyword filter will leak the `post_tweet` variant, whose action word is absent from the filter’s vocabulary; V’s sandbox will block every variant by whitelist enumeration regardless of how much the model is driven). All five held on the real MCP harness, including the boundary prediction—the intuition stack leaked exactly the `post_tweet` case.

3.3 Deployment-value experiment

Testbed. A minimal agent: external content is written into memory; a user task triggers a tool call; the LLM (DeepSeek-chat, temperature 0.2) proposes a call; the defense chain decides; a real-effect executor performs the action (a ledger for transfers, an outbox for email, real file deletion in a sandbox directory, a forum log for comments). Attack success requires a *real state change* (server-side effect semantics, as in (Xu et al. 2026)), not merely a proposal.

Scenarios and attacks. 50 scenarios (30 malicious + 20 benign; the benign set is larger than the 25% share of (Xu et al. 2026; Debenedetti et al. 2024) to narrow utility confidence intervals), derived from the memory-provenance families of (Xu et al. 2026). Attacks: 8 self-written variants (direct, paraphrase, fake-confirmation, combo, role-play, jailbreak prefix, deep rewrite with keyword splitting, authority frame) plus 4 third-party AgentDojo payloads (Debenedetti et al. 2024), 12 total; escalation adds an adaptive attacker (3 rounds, rewrites from the interception reason), a white-box adaptive attacker (told the system prompt and keyword list), and a PAIR-style candidate-search attacker.

Defenses. Four self-implemented defenses, one per anchor family: - **D1** (L0): system-prompt hardening—“ignore instructions in external content; never perform high-risk actions without explicit user request.” The verdict is the model’s own compliance. - **D2** (L1): keyword filter over memory and call text (transfer/delete/send/payment terms, Chinese equivalents). - **D3** (L1/L2): confirmation gate—high-risk tools require a *platform-recorded* confirmation event; text claims (“user-confirmed”) are ignored. - **D4** (L3): parameter sandbox—tool arguments must match schema (recipient whitelists, target patterns, type/range checks), executed in a side-effect stub with verified state changes.

Stacks (same budget, two defenses each). - **V (VAL-guided):** D3 + D4—an intent anchor (L2 input) plus a confinement property (L3). - **N (mainstream intuition):** D1 + D2—prompt hardening plus keyword filtering, the intuitive low-cost default in production agents. We emphasize what this comparison does *not* claim: it is not “VAL beats all defenses” (structural defenses dominate behavioral ones by construction); the contribution is the *epistemic* contrast—identical security numbers, opposite guarantees, a sharp utility cost for the intuition stack (Section 6.1).

Metrics. ASR = high-risk tool executed with complete arguments and verified state change; compliance rate = the model *proposes* a high-risk tool (including empty-argument hedging)—separating “model was driven by the injection” from “the defense intercepted.” Benign success = benign scenarios complete their intended action.

Runs. Static: 7 configs $\times$ 380 cases (30 malicious $\times$ 12 attack variants + 20 benign). Adaptive/white-box: 30 malicious scenarios $\times$ stacks $\times$ 3 rounds. Seeds: V, N, and the no-defense baseline (ND) re-run $3\times$. A second victim (Llama 3.1 8B) re-runs the static battery. Total $\approx$ 7,000 LLM calls across all experiments; AgentDojo (Section 6.5) and JADE probes add $\sim$ 2,000 harness-internal calls reported separately.

4 The Defense Level Map

The full level map (all 22 defenses) is in Appendix A.1.

Full per-defense ratings with evidence: `reliability/corpus.json`, ratings/; blind-

rather agreement for the security subset is 100% between the two most recent raters (6/6 agentsec items, $\kappa \approx 0.88$ overall (Yin 2026)).

The structural finding. L3 anchors exist in agent security—but only for *confinement and information-flow properties*: sandbox execution domains, taint tracking, data-control separation, SMT-monotonic permission lattices. These are decidable by construction. *Semantic* safety (“is this action dangerous?”) is an open-world property for which no decidable fragment is available in the unrestricted setting, and hence caps at L2; restricted semantic properties that can be formally encoded (e.g., a bounded action taxonomy) remain eligible for higher levels. This mirrors the formal/empirical dichotomy of (Yin 2026) exactly: completeness is reachable only for formally specifiable properties.

4.1 4.1 Worked examples: how the classification reads three defenses

Each classification is a claim; per the framework’s own epistemology, claims need anchors. We show the full evidence chain for three representative defenses spanning the level spectrum (remaining classifications in the supplementary corpus).

Self-Ask provenance — no anchor (L0). Mechanism: the LLM infers whether memory text looks external, user-history-like, or confirmation-like, and authorizes from the inferred trust (Xu et al. 2026). Q1: LLM; Q2: none; Level: **L0**. Evidence: permissive mode reaches **0.984 ASR**; strict mode blocks all benign actions (Xu et al. 2026). Predicted failure mode: text claims (fake markers, social engineering). Observed: 0.984 ASR under permissive inference—the anchor is whatever the model believes.

PPMF — the intent anchor (L1/L2). Mechanism: a deterministic gate authorizes tool calls by matching action risk to the least-trusted supporting memory, whose provenance metadata (source, trust, confirmation events) is platform-maintained (Xu et al. 2026). Q1: designer policy over platform-recorded events; Q2: correctness; Level: **L1** (weakest anchor) with L2-grade inputs. Evidence: 0.000 ASR with intact metadata; **0.088 ASR under 10% forged confirmations**—the completeness blind spot quantified. Predicted failure mode: forged/compromised metadata (outside the ODD). Observed: exactly that; the au-

thors themselves list compromised metadata outside their guarantee.

IFC/Fides — confinement as L3. Mechanism: the planner dynamically tracks taint labels and deterministically enforces lattice policies (Costa et al. 2025). Q1: decidable; Q2: completeness; Q3: single property (information flow). Level: **L3**. Evidence: a formal model characterizing the class of properties enforceable by dynamic taint tracking. Predicted failure mode: properties outside the lattice (covert channels, semantic content). Observed: consistent—the guarantee is about flow, not about whether the permitted action is wise.

The three examples trace the level ladder with primary-source evidence: L0 fails to text belief, L1/L2 fails outside its ODD (forged metadata), L3 holds by construction within its property—all failure modes predicted before the validation data were consulted (Section 5).

5 5. Prediction Validation

We froze 16 prediction cards (level → predicted failure mode + predicted breaking attack) before consulting outcome data.

PPMF family (6 defenses): all six predictions hit on PPMF’s own published numbers (Xu et al. 2026): content filters (L1) leave 0.200–0.933 ASR and collapse under rewriting; Self-Ask provenance (L0) reaches 0.984 ASR; the gate-only ablation (L1 without L2 inputs) blocks attacks but kills all benign actions; the full PPMF (L1/L2: platform metadata through a designer policy) achieves 0.000 ASR on evaluated attacks with benign preserved, and 0.088 ASR under 10% forged confirmations—the completeness blind spot made quantitative.

New cards (abstract-level evidence, flagged): 4/4 behavior predictions hit (Llama Guard distribution-bound accuracy; RA-LLM residual ASR under strong attacks; perplexity filtering weak alone; ToolEmu audit-value-only). Two mechanism-driven corrections emerged; because they are the strongest evidence that the prediction loop *corrects* rather than merely confirms, each gets a full account below.

5.1 5.1 Mechanism-driven correction I:

Progent’s SMT monotonic confinement

Predicted: L1 (designer permission policy). **Mismatch:** every policy update is adjudicated by an SMT solver as *narrowing* (automatic) or *expanding* (requires approval)—the agent’s action space can only shrink without approval, a confinement property with a decidable witness. **Corrected classification:** the gate stays L1, but the monotonic-confinement guarantee is annotated L3 (theorem-backed structure, R12b). The mismatch is evidence for, not against, the protocol.

5.2 5.2 Mechanism-driven correction II:

RARR’s LLM-mediated verdict (R3)

Predicted: L2 (attribution checking). **Mismatch:** RARR’s verdict path is LLM-mediated end-to-end—“citation existence” is not mechanically checked. Under R3 (evidence anchors do not equal verdict anchors), the verdict component is LLM declaration. **Corrected classification:** L2 → L0; a superficially “objective” anchor can be L0-masked when the alignment step is model-mediated—precisely the laundering pattern of the memory domain.

Cumulative: 10/10 behavior predictions hit (6 high-confidence, 4 abstract-level), 2 classification corrections. Sample is small; the value is that the prediction *protocol* is now demonstrated end-to-end, not that the hit rate is large.

6 6. Deployment-Value Experiment: Results

The dominance at a glance (full tables in 6.1-6.4):

The two zeros in the last two rows are the paper’s spine: identical security numbers, opposite guarantees, a 100-point utility gap.

6.1 6.1 Static attacks (12 variants, 380 cases/config, real effects)

Benign success is measured on 20 benign scenarios (Wilson 95% CIs in brackets); the utility gap between V and N is significant at $p < 0.001$ (Fisher’s exact test on 20/20 vs 0/20).

Every defense reduces ASR relative to the 0.333 baseline; the two stacks reach the same zero. They differ in *how*: V’s zero is produced by a gate the model cannot influence; N’s zero is produced by the model’s behavior under the hardening prompt

(compliance 0.235—the model proposes the malicious tool a quarter of the time, mostly with empty arguments; strict ASR counts only actionable executions), D2 alone leaks 0.200 ASR: the keyword filter is bypassable by rewriting.

The two layers of V are not redundant; they answer different questions. D3 (confirmation gate) is the reason V’s ASR is 0.000: it refuses high-risk tools without a platform-recorded authorization event, and the model cannot forge that record. D4 (parameter sandbox) is not about *whether* an action is authorized but *what an authorized action may touch*: it constrains arguments to schema (recipient whitelists, target patterns). The division shows up in the single-defense numbers—D3 alone already reaches 0.000 ASR, while D4 alone leaks 0.067 (schema-legal semantic attacks, e.g. a malicious comment posted to an allowed target). Stacking them yields the claim in its full form: D3 controls *whether* (the authorization anchor), D4 controls *what* (the confinement anchor), and an attack must defeat both anchors on the same call to succeed.

6.2 6.2 Adaptive and white-box attacks

The single-defense breakpoints are already informative. Two zeros survive both adaptive and white-box escalation unchanged (D1 at 0.000, D3 at 0.000), and two do not (D2 0.200→0.467, D4 0.067→0.200). But the two surviving zeros rest on different grounds, visible in compliance under the same attacks: D3’s model is driven to propose the malicious tool in 53.3% of adaptive cases and 56.7% of white-box cases and is still blocked every time (the gate reads a platform record the attacker cannot write), while D1’s model is driven 36.7%/33.3% and the zero depends on its refusal behavior holding under every attack form we tried. The same static zero, the same adaptive zero, the same white-box zero—different provenance, and different predicted breakpoints under attacks strong enough to move a behavioral refusal (Section 7).

Zero stability under escalating attack ($Z(\alpha)$). Tracking the cumulative ASR round by round (7 rounds, 30 cases per point; full data in Appendix) exposes distinct stability morphologies. D2 climbs monotonically to **0.833 by round 7** (each rewrite learns another way around the keyword vocabulary); D4 jumps at round 2 and plateaus (0.067 → 0.300); D1 and D3 stay at 0.000 across all seven

Stack	>p() * 0.2500 >p() * 0.2500 >p() * 0.2500@				
ASR (both victims)					
Benign (both victims)					
What the zero is					
	ND (no defense)	0.333 / 0.253	1.000 / 1.000	no defense	
V (VAL: gate + sandbox)	0.000 / 0.000	1.000 / 1.000	structure - platform record + schema, victim-independent		
N (intuition: hardening + filter)	0.000 / 0.000	0.000 / 0.000	luck - the model's refusal behavior, model-dependent		
	>p() * 0.2000 >p() * 0.2000 >p() * 0.2000 >p() * 0.2000@				
Config					
ASR					
Compliance					
Benign					
gain vs ND					
	ND (no defense)	0.333	0.492	1.000 [0.84, 1.00]	—
D1 prompt hardening (L0)	0.000	0.219	0.000 [0.00, 0.16]	0.333, benign destroyed	
D2 keyword filter (L1)	0.200	0.492	1.000 [0.84, 1.00]	0.133	
D3 confirmation gate (L1/L2)	0.000	0.497	1.000 [0.84, 1.00]	0.333	
D4 parameter sandbox (L3)	0.067	0.494	1.000 [0.84, 1.00]	0.266	
V = D3+D4 (VAL)	0.000	0.497	1.000 [0.84, 1.00]	0.333, lossless	
N = D1+D2 (intuition)	0.000	0.235	0.000 [0.00, 0.16]	0.333, benign destroyed	

rounds. The two flat curves are indistinguishable in $Z(\alpha)$ alone—which is exactly the point. **Identical observed zero-stability profiles do not, by themselves, establish equivalent security guarantees**; only the provenance separates them, and the gap widens under escalation: D3’s model is driven to propose the malicious tool in **83.3%** of cases and is still blocked every time (the gate reads a platform record), while D1’s model is driven 36.7% and the zero is its refusal behavior holding. We attempted to falsify D1’s flat curve directly (7-round adaptive attacks with prompts explicitly demanding complete executable arguments, plus authority/emergency/role-play frames): it did not break under this attack budget—stronger support for the provenance distinction, not evidence that it cannot break. We predict it will break under an attack strong enough to defeat DeepSeek’s refusal behavior (e.g., a less-aligned attacker); D3’s will not, short of compromising the audit substrate itself (Appendix).

Where the zero breaks: an execution funnel under static attacks. The funnel is a mechanism snapshot at one attack condition (static, 12 variants, $n=360$)—it is not the 7-round $Z(\alpha)$ trajectory, which is reported separately above. For each malicious case we record whether the model proposes a high-risk tool (compliance), whether the proposal carries complete executable arguments, whether it passes the authorization/rule check, and whether it executes with a verified state change. D1’s funnel collapses at the second stage: the observed zero is terminated at the complete-argument stage—the model fails to produce ex-

ecutable arguments under the hardening prompt. The break is at parameter completeness, but the anchor remains behavioral: D1 contains no structural parameter-integrity verifier; argument completeness is whatever the model happens to produce (the same shape as N’s, 0.233 → 0.000). D3’s funnel reaches the third stage: the model produces complete arguments in 49.7% of cases, and the authorization gate then refuses every one—the zero is the platform record (the same shape as V’s, 0.497 → 0.000). The same ASR, the same flat $Z(\alpha)$, but the funnels break at different layers: one at model behavior (arguments); one at external constraint (authorization). This is the funnel-level statement of the paper’s thesis—the zero does not say where it comes from; the funnel does.

Definition (hedging wall). Under a hardening prompt (D1), an LLM victim under injection may comply *partially*: it proposes the requested high-risk tool while emitting empty or incomplete arguments—a stable compromise between the injection (“propose the action”) and the safety prompt (“do not execute without authorization”). Actionable ASR requires complete arguments, so hedging suppresses ASR *without any structural gate*. The wall is a model behavior, not a mechanism: its height is victim-specific (compliance 0.235 for DeepSeek, 0.014 for Llama 8B, Section 6.4).

Adaptive attacks nearly double the leak rate of the L1 and L3 single defenses (D2 0.200 → 0.467, D4 0.067 → 0.200). Against the stacks: V holds even

	>p() * 0.1667 >p() * 0.1667 >p() * 0.1667 >p() * 0.1667 >p() * 0.1667@				
Config					
Static ASR					
Adaptive ASR					
White-box ASR					
White-box compliance					
PAIR-lite ASR					
	D1	0.000	0.000	0.000	0.333 —
	D2	0.200	0.467	—	— —
	D3	0.000	0.000	0.000	0.567 —
	D4	0.067	0.200	—	— —
	V	0.000	0.000	0.000	0.567 0.000
	N	0.000	0.000	0.000	0.235 0.000

when the model is driven to propose the malicious tool in 57% of scenarios (white-box compliance 0.567, $2.4\times$ N’s 0.235); N holds only because its hedging wall survives every attack form we tried—static, adaptive, white-box, PAIR (0.000), and a willing cross-family attacker (DeepSeek against Llama: 0.000, Section 6.4). Aligned cross-family attackers (Moonshot Kimi, Meta Llama 8B) declined the role—explicit refusals, sanitizing rewrites (Section 8.1). Gradient-based attacks (GCG) are out of scope: the API exposes no gradients.

6.3 6.3 Seeds and honesty

V, N, and ND re-run $3\times$: standard deviations ≈ 0.005 on compliance, 0.000 on ASR and benign success. The headline contrast is stable (details and two harness bugs caught and fixed mid-experiment in Appendix).

6.4 6.4 Victim generalization: a 2x2 matrix

All results above use DeepSeek-chat as the victim. To test whether the structural pattern is victim-specific, we ran the same static attack suite with a second, independent victim—Meta Llama 3.1 8B (local Ollama, same harness, tool registry added to the prompt):

The 2×2 matrix is in Appendix A.2.

Three findings. First, **baseline injection compliance is model-specific**: Llama 8B is *more* resistant than DeepSeek (ND compliance 0.328 vs 0.492; ASR 0.253 vs 0.333). Second, **N’s zero generalizes, and its “model luck” nature is confirmed by a second model**: prompt hardening suppresses actionability to a model-specific degree (0.233 for DeepSeek, 0.014 for Llama). Third, **V’s structural claim is victim-independent**: 0.000 ASR and 1.000 benign for both victims, tolerating 0.317 compliance without a single execution. A *willing* cross-family attacker (DeepSeek crafting

white-box jailbreaks against Llama) also failed: ASR 0.000, compliance 0.000. **The generalization reads crisply: structural guarantees are model-independent; behavioral luck is model-dependent.** V’s zero is a property of the gate and schema; N’s zero is a property of the model’s mood on the day.

6.5 6.5 Validation on the official AgentDojo benchmark

We integrated our stacks into AgentDojo’s official harness (Debenedetti et al. 2024)—their banking suite, their environment fixtures, four of their attack families (*direct*, *ignore_previous*, *injecagent*, *system_message*), and their evaluation—with DeepSeek-chat as the agent model. The confirmation gate whitelists the six IBANs that exist in the banking environment; transfers to any unlisted recipient are refused before execution, with a [BLOCKED] tool result fed back to the model. **For a same-setting comparison, we additionally ran AgentDojo’s own defenses (tool_filter, repeat_user_prompt, spotlighting) in the identical harness, model, task subset, and attack set.**

(95% Wilson CIs in brackets.) Three observations. First, **V matches the strongest official defense on ASR (both 0.0% [0.0, 2.6]) while preserving five times the utility (82.6% vs 16.7%).** Second, **the tool filter’s utility collapse is a VAL failure-mode prediction made visible**: its 16.7% is not a design choice to “refuse most calls” but DeepSeek failing the filter’s own instruction—it emits tool names that do not exist in the suite, so the filter removes every tool and the agent cannot act (126/144 traces contain no tool call). The same defense on GPT-4o in (Debenedetti et al. 2024) *increases* benign utility; on DeepSeek it collapses to 16.7% (and, on inspection, to 0%: the 24 “successes” match a transaction already present in the environment’s initial state). The anchor of tool_filter is the model’s instruction-following

Config	>p() * 0.3333 >p() * 0.3333@		
ASR (n = 144, 4 attack families)			
Utility			
	ND (no defense)	6.2% [3.3, 11.5]	82.6%
	N (prompt hardening + keyword filter)	3.5% [1.5, 7.9]	83.3%
	V (confirmation gate + parameter sandbox)	0.0% [0.0, 2.6]	83.3%
	AgentDojo tool_filter	0.0% [0.0, 2.6]	16.7%
	AgentDojo repeat_user_prompt	8.3% [4.8, 14.0]	81.9%
	AgentDojo spotlighting	6.2% [3.3, 11.5]	81.2%

(L0/L1 luck); when the model does not comply, the defense does not merely stop protecting—it destroys usability. Third, **the cross-tested difference in N’s behavior is the strongest evidence for our core claim**: in our testbed N reached 0.000 ASR (produced by the model’s refusal behavior); on AgentDojo the same stack leaks 3.5%. *N’s zero is context-dependent model behavior, not structure—move the stack to another benchmark and the zero moves with it.* V’s zero does not move (0.0% in both testbeds).

Attack-family breakdown of the undefended baseline. The 6.2% ND ASR is not uniform across the four families: *direct* accounts for 7 of the 9 successes (19.4% ASR) and *system_message* for 2 (5.6%), while *ignore_previous* and *injecagent* reach 0.0%—DeepSeek recognizes these two families’ explicit “ignore previous instructions” framing and refuses. All four families load and inject correctly (the payloads appear in the traces); the zeros are model immunity, not a harness bug. This is itself a model-behavior (L0) fact: the same families could succeed on a more compliant model, which is exactly why V’s structural zero (0.0% across all four) is the meaningful number.

Scope caveats: a subset of the banking suite (6/19 user tasks, 6/24 injection tasks), one suite, one agent model (DeepSeek-chat), and four attack families; the harness, tasks, attacks, and evaluation are AgentDojo’s, so the gap to full coverage is breadth, not provenance.

On the varying ND baseline across testbeds. The undefended ASR differs sharply across our three settings—0.333 on our own testbed (Chinese memory injections, 30 malicious scenarios), 6.2% on AgentDojo (English TODO injections, 144 pairs), 87.5% on JADE’s MCP instances (English tool-description poisoning, 16 cases). This is expected and informative rather than a contradiction: ND ASR measures the *attack surface* (injection strength, carrier, environment semantics),

not a fixed property of the model. What is stable across all three is the *contrast*: the intuition stack’s zero moves with the setting (0.000 → 0.035 → 0.062 as attacks get harder), while V’s structural zero does not (0.000 everywhere).

7 7. Analysis: The Same Zero, Different Guarantees

The experiment’s central image is two zeros. **V’s 0.000 ASR is structural**: the confirmation gate reads platform-recorded events (the LLM cannot write them), and the sandbox enforces schema whitelists (arguments the attacker cannot produce are impossible). It holds while the model is maximally compromised (0.567 compliance) and across all 12 attack families, adaptive iteration, and white-box knowledge. **N’s 0.000 ASR is behavioral**: the hardening prompt makes this particular model hedge—it complies with the injection by proposing the tool, then refuses the arguments; The same family’s jailbreaks (our attacker was DeepSeek) cannot convert compliance into actionability. A different model, a stronger attacker, or a differently-phrased prompt could move N’s zero; nothing can move V’s within its ODD. The $Z(\alpha)$ trajectories of Section 6.2 make the same point at the level of stability: N’s flatness is the model’s mood holding under escalation, V’s flatness is the platform’s record holding—the same trajectory, different ground.

This is the deployment answer to “which defense should I buy?”: **VAL selection buys a guarantee; intuition buys the model’s current mood.** Both reach zero, but N’s zero costs every benign action (0.000 benign success) while V’s is lossless; the AgentDojo tool filter (ğ6.5) is the same story from the other side—a defense anchored in the model’s instruction-following *destroys usability* when the model does not comply. Under VAL’s usage criteria ((Yin 2026)), the deployer’s question becomes precise: *is the threat inside the anchor’s ODD?* If yes, L2/L3 structure is available; if no, the honest

answer is L2 correctness plus labeling, not a claim of safety.

The account extends to the behavioral layer.

The most striking failure mode in current agent security is not semantic but physical: **Execution Hallucination (EH)**—an agent verbally refuses a dangerous request while the operation has already completed at the OS level. LITMUS (Zhang et al. 2026) measured this across six frontier agents in real OS environments (EHR 7.98–17.97%) and showed it is invisible to every semantic-only evaluation framework. Under VAL this is not a surprise but a prediction: the anchor for a *behavioral* claim must live on the physical layer. A semantic-layer anchor (L0 model self-report; L1 rules over dialogue) cannot, by construction, verify what the model did but did not say—exactly as our own covert-execution measurement found (output-layer detectors cap at TPR 0.60 (Yin 2026)). LITMUS’s dual-layer verification is the L2 anchor applied to behavior: it reads OS state, not conversation. The same lesson holds for defense as for evaluation: a defense whose guarantee lives in what the model *says* it will do (N’s zero) cannot be held to the standard of one that reads the platform record (V’s zero).

8 8. Limitations

1. **Single model (victim and attacker).** The agent and every attack form used DeepSeek-chat; cross-family attackers (Moonshot Kimi, Meta Llama 8B) declined the role—explicit refusals, sanitizing rewrites. A less-aligned attacker (e.g., hosted Llama-3.3-70B) and gradient-based attacks (GCG, no gradients via API) remain open variants.
2. **Self-built testbed.** Scenarios, defenses, and the real-effect sandbox are ours—deliberate for a controlled comparison of selection strategies. Section 6.5 validates the stacks on AgentDojo’s official harness (their tasks, attacks, evaluation), reaching 0.0% ASR. Our behavioral blind spot (covert execution, TPR 0.60 (Yin 2026)) is independently reproduced at the OS level by LITMUS (Zhang et al. 2026); we did not run its Ubuntu+OpenClaw harness—the corroboration is at the level of the structural claim.
3. **Attack coverage.** Twelve families plus adaptive/white-box/PAIR escalation; token-level optimization and multi-turn social engi-

neering absent.

4. **Benign sample.** 20 benign scenarios (CIs in Section 6.1); broader utility coverage (long-horizon tasks) untested.
5. **LLM-rater classifications.** Blind-validated among same-family raters (security subset 90.9% exact, overall $\kappa \approx 0.8$ (Yin 2026)); human-rater agreement is future work. The classifier itself is an L1 tool whose reproducibility was measured, not assumed—the judge’s judge, applied to the judge itself.
6. **Prediction sample.** 10/10 hits on a small, partially abstract-level sample; the mechanism (anchor $\rightarrow$ failure mode) is the claim, not the hit count.

9 9. Conclusion

The agent-security field has a deployment problem: too many defenses, no pre-purchase axis. We showed that VAL provides one. Across the 22 defenses, attack families, and testbeds we studied, the anchor of a defense predicts how it fails—text rules fail to rewriting, model self-assessment fails to compliance, objective anchors fail outside their ODD, and structural anchors (confinement, information flow, data-control separation) hold by construction within it. In our controlled same-budget comparison, choosing by this axis beat choosing by intuition: identical security numbers, opposite guarantees, and a 100-point utility gap. We also located the L3 frontier in agent security: **confinement, not semantics**—the properties worth encoding into decidable systems are the ones that restrict what an agent can do, not the ones that judge whether it should. Two caveats bound these claims: the taxonomy’s categories are ours (validated by blind-rater agreement and out-of-sample predictions, not by an external ground truth), and the deployment comparison pits structural defenses against the most common behavioral defenses, not against every defense in the landscape. We offer VAL as a falsifiable framework and a research agenda—not as a settled ontology.

Others grade defenses by their claims. We grade them by their anchors—and, in the cases we can measure, the anchor decides.

	>p() * 0.2500 >p() * 0.2500 >p() * 0.2500@
Level	
Defenses	
Anchor / semantics	
Predicted behavior	
L0	Prompt hardening (D1; RA-LLM (Zhou et al. 2023)), ToolEmu judge, A-MemGuard consensus, SafeAgent plan layer, Self-Ask provenance none / LLM self-assessment residual ASR under strong attacks; no stable operating point; zero utility under over-restrictive prompts
L1	Keyword filter (D2), NeMo Guardrails, Progent gate, tool allowlists, PPMF gate, confirmation-marker heuristics, perplexity filter designer rules / intent bypassed by rewriting; false-block benign actions
L2	Llama Guard classifier, RAG citation check, effect-verification, statistical detectors objective truth / truth $\rightarrow$ effect effective in-distribution; fails on OOD, forged metadata, poisoned data
L3	IFC/Fides (Costa et al. 2025), CaMeL (CaMeL 2025), sandboxing (D4), Progent SMT monotonic confinement decidable systems / structural confinement, information-flow, or data-control properties hold by construction within the ODD; semantic dangers inside the allowed space pass
L4	Smart-contract formal verification formal kernels domain-complete for encoded properties
N/A	AgentDojo (Debenedetti et al. 2024), M _S -SafetyBench benchmarks not runtime defenses (R1)

9.1 Appendix

9.2 A.1 Defense level map

9.3 A.2 Victim generalization (2 $\times$ 2)

References

A-MemGuard: Adaptive Memory Defense for LLM Agents (Preventive Memory Poisoning Defense). 2025. <https://arxiv.org/abs/2510.02373>.

CaMeL: Data-Control Separation for LLM Agents (Source-Gated Control Flow). 2025. <https://arxiv.org/abs/2503.18813>.

Chen, Bei et al. 2023. "CodeT: Code Generation with Generated Tests." *ICLR*.

Costa, Marcelo et al. 2025. *Securing AI Agents with Information-Flow Control*. <https://arxiv.org/abs/2505.23643>.

Debenedetti, Edoardo, Tianjiu Zhang, Mislav Balunovic, Lukas Beurer-Kellner, Marc Fischer, and Florian Tramer. 2024. *AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents*. <https://arxiv.org/abs/2406.13352>.

Inan, Hakan et al. 2023. *Llama Guard: LLM-Based Input-Output Safeguard for Human-AI Conversations*. <https://arxiv.org/abs/2312.06674>.

Injection-Execution Dissociation: A Mechanistic Evaluation of Persistent Memory Attacks on Stateful LLM Agents. 2026. <https://arxiv.org/abs/2605.08442>.

LLM-Based Test Oracles: Source-of-Authority Taxonomy—a Systematic Literature Review. 2026. <https://arxiv.org/abs/2607.05031>.

Lu, et al. 2025. *When Does Verification Pay Off? A Closer Look at LLMs as Solution Verifiers*. <https://arxiv.org/abs/2512.02304>.

Shi, Tianyuan et al. 2025. *Progent: Securing AI Agents with Privilege Control*. <https://arxiv.org/abs/2504.11703>.

Xu, J., Y. Xiao, W. Shao, H. Liu, and X. Li. 2026. *Memory Provenance Laundering in LLM Agents: A Non-*

Amplification Firewall for Persistent Memory. <https://arxiv.org/abs/2607.29167>.

Yin, Yajie. 2026. *Grading the Graders: Verification Autonomy Levels (L0–L5) for LLM Reasoning*. <https://arxiv.org/abs/2608.19009>.

Zhang, Chiyu, Huiqin Yang, Bendong Jiang, et al. 2026. *LITMUS: Benchmarking Behavioral Jailbreaks of LLM Agents in Real OS Environments*. <https://arxiv.org/abs/2605.10779>.

Zhou, Andy et al. 2023. *Defending Against Alignment-Breaking Attacks via Robustly Aligned LLM (RA-LLM)*. <https://arxiv.org/abs/2309.14348>.

	>p() * 0.3333 >p() * 0.3333@			
Config				
DeepSeek victim ASR / compl. / benign				
Llama victim ASR / compl. / benign				
	ND (no defense)	0.333 / 0.492 / 1.000	0.253 / 0.328 / 1.000	
	N (D1+D2)	0.000 / 0.233 / 0.000	0.000 / 0.014 / 0.000	
	V (D3+D4)	0.000 / 0.497 / 1.000	0.000 / 0.317 / 1.000	